

AI Workflow Handoff for Developers

Purpose

This document is a practical handoff for anyone who needs to understand, debug, or extend the AI-driven quiz workflow in this app.

The app is not a simple one-shot chatbot. It runs a multi-stage pipeline that combines:

- a Tkinter front end
- a local topic library under X/
- AI agents invoked through the opencode CLI bridge
- parsing and validation layers that turn raw AI responses into structured quiz data

If you are trying to understand how the app “thinks,” this is the map to follow.

1. High-level flow

The workflow is centered around the quiz experience and roughly follows this path:

1. User opens the Quiz page and enters the chat stage.
2. The app loads selected topic folders from the local library.
3. The pre-quiz chat uses the main chat agent.
4. Background models analyze the topic context and produce reports.
5. A quota planner agent turns those reports + user intent into a structured quota manifest.
6. A topic resolver and fallback builder convert that plan into file query specs.
7. The app retrieves candidate questions from the bank.

8. A blueprint is generated and parsed.
9. Non-sim and sim questions are assembled into a final quiz question list.
10. The quiz runs, and answers can be evaluated or summarized by AI.

In short: chat -> reports -> plan -> retrieve -> build -> quiz -> evaluate.

2. Main entry points

UI entry

- [main_app/ui/quiz_page.py](#)

This is the main orchestrator for the quiz lifecycle. It owns the state machine:

- CHAT
- THINK
- LOAD
- QUIZ
- DONE

It decides which screen is shown, handles back/cancel behavior, and launches the relevant pipeline phases.

App bootstrap

- [main_app/app.py](#)
- [main_app/main.py](#)

These are the startup layers. The app is launched from the module entry point and then boots the Tkinter window.

Settings and provider wiring

- [main_app/ui/main_window.py](#)
- [main_app/config.py](#)
- [main_app/api_client.py](#)

These files control how the app connects to AI providers and whether it uses direct API calls or the CLI bridge.

3. The AI execution stack

A. Chat and background report generation

Primary file: - [main_app/agent_runner.py](#)

This is the central AI runner. It manages:

- main chat calls
- background model calls
- session persistence per agent
- pending/retry behavior for network issues
- the later quota-planner and query-specifier pipeline

Key agent names used by the app include:

- pre-quiz_main_chat
- diagnostic-quota-planner
- agent2a-topic-resolver
- agent2c-fallback-builder
- knowledge-graph-builder
- mechanism-evaluator
- misconception-classifier
- drill-recommender

Important detail: the app uses the opencode CLI bridge, not direct prompt calls for the agent-driven workflow. That means the app depends on the local opencode setup being available and the agents being registered.

B. API client layer

- [main_app/api_client.py](#)

This file handles low-level model calls. It supports:

- direct HTTP API calls for GROQ/OpenRouter/custom providers
- a CLI bridge client for opencode

The CLI bridge client is the crucial piece for the agent workflow.

C. Quiz evaluation and tutoring

- [main_app/evaluation_runner.py](#)

This file bridges the AI into the quiz runtime. It covers:

- pre-quiz chat
- live evaluation during the quiz
- batch evaluation at the end
- summary report generation
- tutoring thread responses after the quiz

If a bug appears in AI feedback, summary text, or tutor replies, this is one of the first modules to inspect.

4. The workflow in order

Stage 1: Chat setup

Triggered from: - [main_app/ui/quiz_page.py](#)

Flow: - selected topics are loaded from the local library - the chat screen is shown - the user talks about the exam goal, topic focus, and question preferences

The main chat agent is called from [main_app/agent_runner.py](#).

Stage 2: Background analysis

Also driven by [main_app/agent_runner.py](#).

The app sends the student's message to four background agents:

- knowledge graph builder
- misconception classifier
- mechanism evaluator
- drill recommender

These reports are collected and later used by the quota planner.

Stage 3: Quota planning

Handled by the quota planner agent in [main_app/agent_runner.py](#).

This step converts the user request and the background reports into a structured manifest that describes:

- question count
- slot allocations
- flaw priorities
- topic emphasis
- style constraints

The manifest is validated before the next stage continues.

Stage 4: Agent 2 query specification

Still in [main_app/agent_runner.py](#).

This is the “AI workflow” part that resolves the abstract plan into real bank queries.

It runs a two-step sequence:

1. topic resolver agent
2. fallback builder agent

The code also performs tag mapping locally before the fallback builder is called.

This stage produces a file query spec manifest that says which subject/topic/type/format combinations should be searched.

Stage 5: Candidate retrieval

- [main_app/file_retriever.py](#)

This layer takes the manifest and the loaded topic files, then retrieves candidate questions from the available question banks.

The retrieval logic is important because it determines what the downstream AI sees as its pool of possible questions.

Stage 6: Blueprint generation

- [main_app/blueprint_generator.py](#)

The blueprint is a structured plan for the quiz. It describes which question slots are:

- MCQ or subjective
- simulation or non-simulation
- associated with a topic

This is parsed from an AI response and then later used to organize the actual question content.

Stage 7: Question generation and answer matching

- [main_app/question_generator.py](#)
- [main_app/answer_matcher.py](#)

This is where the app turns the blueprint and retrieved questions into the final quiz content.

Key behaviors:

- non-sim questions are filled from matched bank content
- sim questions are filled from simulation sets when present
- MCQ options are parsed and cleaned
- the final list is assembled in order for the actual quiz UI

Stage 8: Quiz runtime and evaluation

- [main_app/ui/quiz_page.py](#)
- [main_app/evaluation_runner.py](#)

This is the runtime surface where the generated quiz is shown to the student. It also triggers:

- live evaluation
 - batch evaluation
 - summary generation
 - tutoring follow-up
-

5. Where to look when debugging

If the app fails before the quiz starts

Inspect: - [main_app/ui/quiz_page.py](#) - [main_app/agent_runner.py](#) - [main_app/config.py](#)

Common issues: - no topics selected - opencode not installed - agents not registered in the opencode config - bad topic folder structure under X/

If the main chat is not responding

Inspect: - [main_app/agent_runner.py](#) - [main_app/api_client.py](#)

Check: - whether the CLI bridge client is active - whether the main chat agent is registered - whether session persistence is interfering

If the blueprint looks wrong

Inspect: - [main_app/blueprint_generator.py](#) - [main_app/question_generator.py](#)

The app assumes very specific AI response formats. If the model output changes, the parsers may silently skip content.

If the candidate pool is empty or wrong

Inspect: - [main_app/agent_runner.py](#) - [main_app/file_retriever.py](#) - [main_app/answer_matcher.py](#)

This is often where the “AI planning looked correct but the actual questions were bad” problem shows up.

If evaluation or tutor replies are wrong

Inspect: - [main_app/evaluation_runner.py](#) - [main_app/prompts.py](#) - [main_app/score_parser.py](#)

This is where answer scoring and explanation formatting are handled.

6. Important assumptions and constraints

The AI workflow depends on opencode

The agent-driven workflow does not work unless the opencode CLI bridge is available and the relevant agents are registered.

The app expects structured AI output

Several parsers assume a fairly stable output shape. If the model output drifts, parsing will fail or silently degrade.

Topic data is local and folder-based

The app relies on local content under X/ and expects it to be organized in a recognizable subject/topic structure.

The UI is asynchronous

Many AI calls run in the background and update the Tkinter UI through callback plumbing. This means UI bugs can be caused by race conditions or thread-safety issues.

7. Recommended investigation order

If you are new to this codebase, the fastest path is:

1. Read [main_app/ui/quiz_page.py](#) to see when the workflow starts.
 2. Read [main_app/agent_runner.py](#) to see how the agents are invoked and persisted.
 3. Read [main_app/file_retriever.py](#) and [main_app/question_generator.py](#) to understand how the AI output becomes quiz content.
 4. Read [main_app/evaluation_runner.py](#) to see how AI is used during and after the quiz.
 5. Then inspect [main_app/api_client.py](#) for provider wiring and transport details.
-

8. Useful local debugging notes

Run the app

From the project root:

```
bash python -m main_app
```

Check the agent registration state

The app checks registered agents by reading the local opencode config. If agents are missing, the workflow will fail later in a confusing way.

Watch for validation failures

The workflow has validation layers around the quota manifest and query spec manifest. These are often the first place to inspect when the pipeline stops mid-way.

9. Open-ended areas to explore

If you want to improve the AI workflow next, the most promising areas are:

- improving the reliability of agent response parsing
 - making the fallback logic more robust when no bank questions match
 - tighten validation around blueprint and question-generation outputs
 - make the workflow more observable with richer logging
 - reduce the dependency on fragile prompt formatting and make it more deterministic
-

10. Short version

If you only remember one thing: the AI workflow is orchestrated by [main_app/agent_runner.py](#), the quiz lifecycle is coordinated by [main_app/ui/quiz_page.py](#), and the transformation from AI output into

actual questions happens in [main_app/file_retriever.py](#), [main_app/blueprint_generator.py](#), and [main_app/question_generator.py](#).